pfi

# Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

# SQL With sqlite3

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Week 3 opens: the book in a database

Courses 1 and 2 answered every question about the book with pandas. On many desks the same data sits in a database, and SQL is how you ask it. Today the synthetic book goes into one database file.

**Every query sits beside its pandas version, and an assert proves the two return the same rows.**

**No live cells today: everything comes from the course's files, so no FRED key is needed.**

### Week 3: SQL, AI, and the snapshot

| | |
|---|---|
| **Day 1** | **SQL With sqlite3** |
| **Day 2** | SQL joins, time series |
| **Day 3** | Working with an AI assistant |
| **Day 4** | The daily market snapshot |
| **Day 5** | The capstone |

# Why SQL: you already know most of it

| Job | pandas | SQL |
|---|---|---|
| Pick columns | `holdings[["cusip", "oas"]]` | `SELECT cusip, oas FROM holdings` |
| Pick rows | `holdings[holdings["oas"] > 200]` | `WHERE oas > 200` |
| Sort, first five | `.sort_values(...).head(5)` | `ORDER BY oas DESC LIMIT 5` |
| Group and total | `.groupby("sector").agg(...)` | `GROUP BY sector, COUNT, SUM, AVG` |
| Keep some groups | `a filter after groupby` | `HAVING` |
| Join | `pd.merge(...)` | `JOIN (tomorrow)` |

**Assistants write SQL readily, and it runs whether it is right or not. Read it, then check it against pandas.**

# sqlite3: a database in one file

### One file

The whole database is a file the program reads and writes. No server, no password.

### Built in

Python ships the sqlite3 module: SQLite 3.50.4 here.

### Everywhere

"Likely used more than all other database engines combined."

```
connection.execute("SELECT 7 / 2, 7 / 2.0")
7 / 2 = 3, 7 / 2.0 = 3.5
```

**Two whole numbers divide as whole numbers. Write * 1.0 on a count.**

SQLite, "Single-file Cross-platform Database" and "Most Widely Deployed and Used Database Engine", sqlite.org (read 2026-10-04).

# load_tables: CSV in, counts checked

```
frame = pd.read_csv(csv_path)
frame.to_sql(name, connection, if_exists="replace")
# read the count back and check it
SELECT COUNT(*) FROM {name}
```

A table name cannot be a ? parameter, so only plain names (letters, digits, underscore) are allowed.

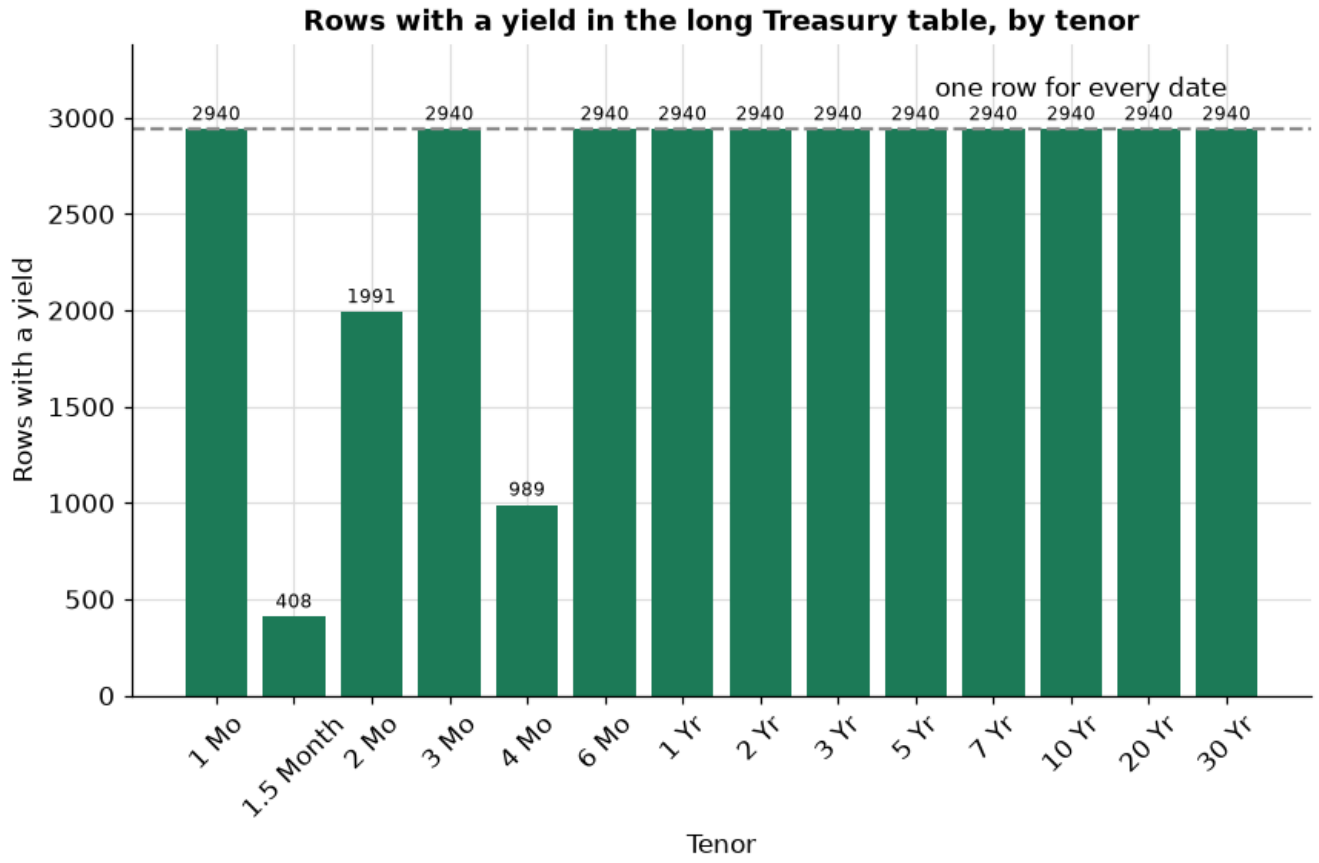
**Seven tables, each count checked**

| | |
|---|---|
| holdings | 200 rows |
| issuers | 150 rows |
| ratings | 18 rows |
| benchmark | 821 rows |
| trades | 374 rows |
| treasury | 35,728 rows |
| spreads | 2,940 rows |

```
b'SQLite format 3\x00'
```

**A database file is binary and rebuilt in one call: *.db stays in .gitignore.**

# Wide to long: what melt leaves out




**41,160**

cells in the wide file: 2,940 dates by 14 tenors

**35,728**

rows with a yield: the long table keeps only these

Late starters: 1.5 Month has 408 rows, 2 Mo 1,991, 4 Mo 989. A missing yield is a missing row, not a blank.

# SELECT, WHERE, ORDER BY, LIMIT

```
SELECT cusip, issuer, market_value
FROM holdings
WHERE sector = ? AND oas > ?
ORDER BY market_value DESC
LIMIT ?
```

### `run_query`

One query, a DataFrame back; values through params; the connection closed.

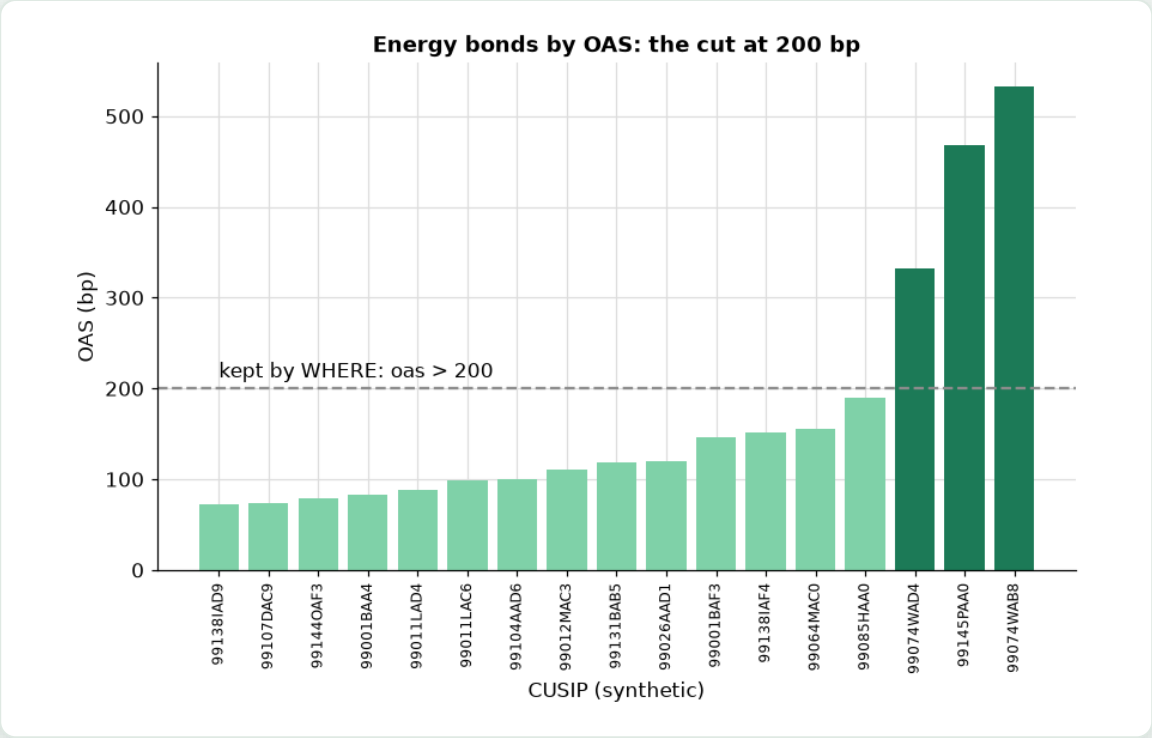
### `assert_same_rows`

Sorted by your keys, index reset; text exact, decimals to rtol 1e-12.

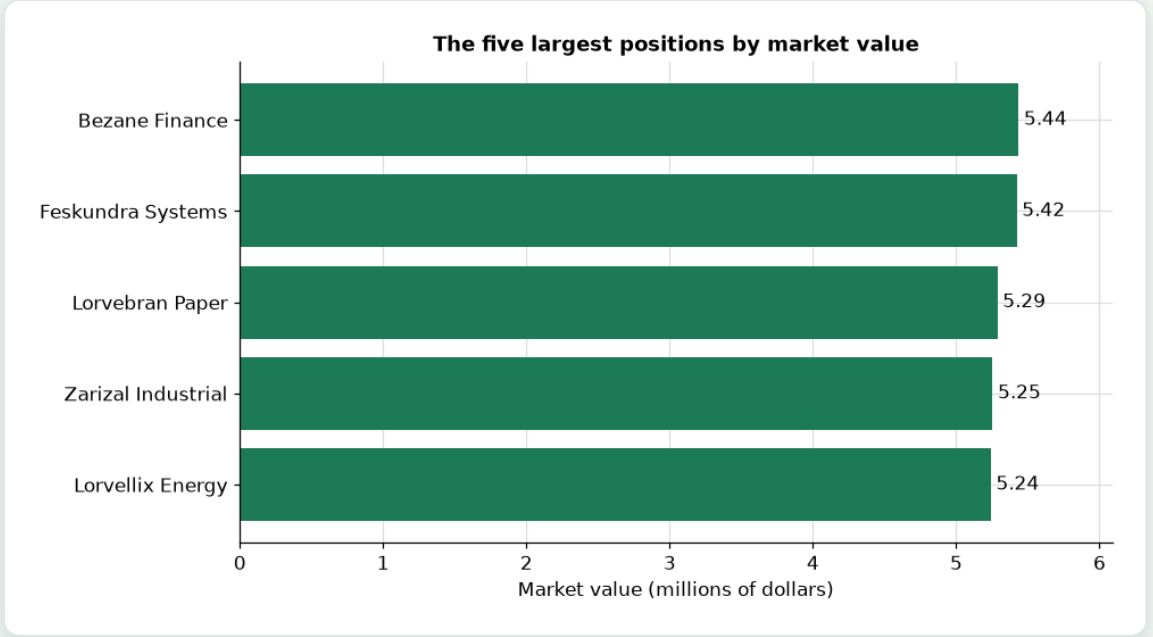
**Without ORDER BY, a database promises no order: LIMIT 5 alone is five rows, not the first five.**

Dates are ISO text, YYYY-MM-DD: year first, so < on text compares in date order. They come back as text; convert with pd.to_datetime.

# What WHERE and LIMIT keep




WHERE keeps the dark bars above the dashed line: the rows the query returned.




ORDER BY ... DESC puts the largest first; LIMIT 5 cuts the list after five.

# Values go in as ? parameters

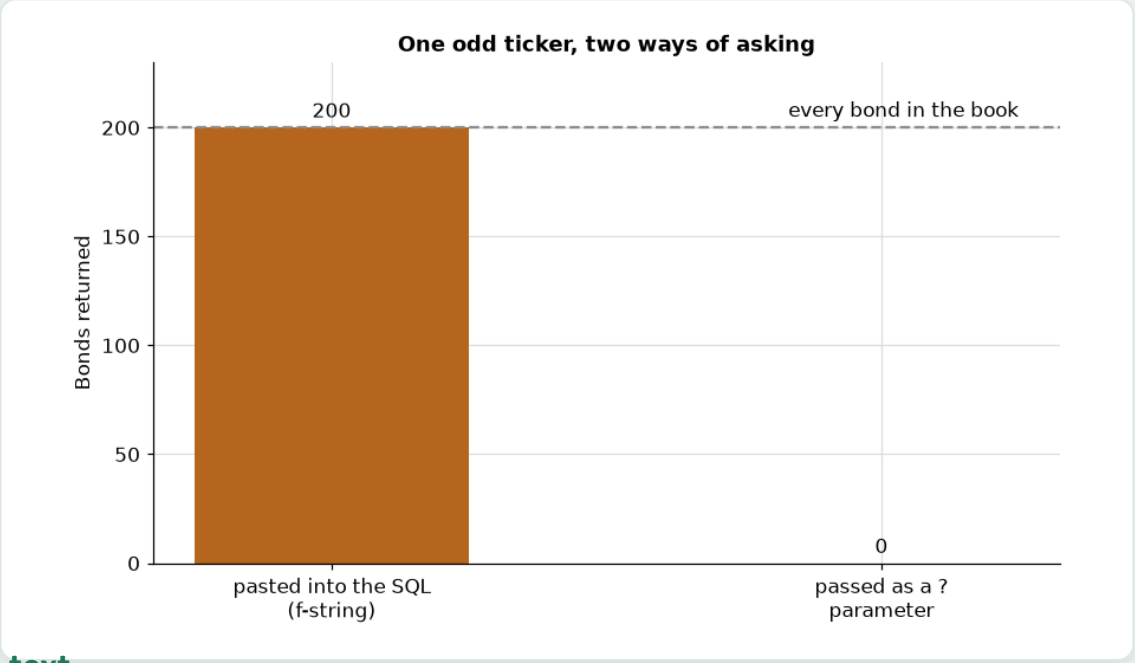
Pasted into the SQL with an f-string, a value becomes SQL: a quote can break the query or change it. A ? parameter is sent separately and is only ever a value.

### A ticker with a quote, pasted: O'BRN

```
DatabaseError: ... near "BRN": syntax error
```

### A value that rewrites the question: NONE' OR '1'='1

```
pasted into the SQL: 200 rows
as a ? parameter:    0 rows
```




**This is SQL injection. Only a checked table or column name ever goes into the text.**

# GROUP BY, then HAVING

```
SELECT sector, COUNT(*) AS bonds
FROM holdings
WHERE oas > ?          -- rows first
GROUP BY sector
HAVING COUNT(*) >= ?   -- then groups
ORDER BY sector
```

**The clauses, always in this order**

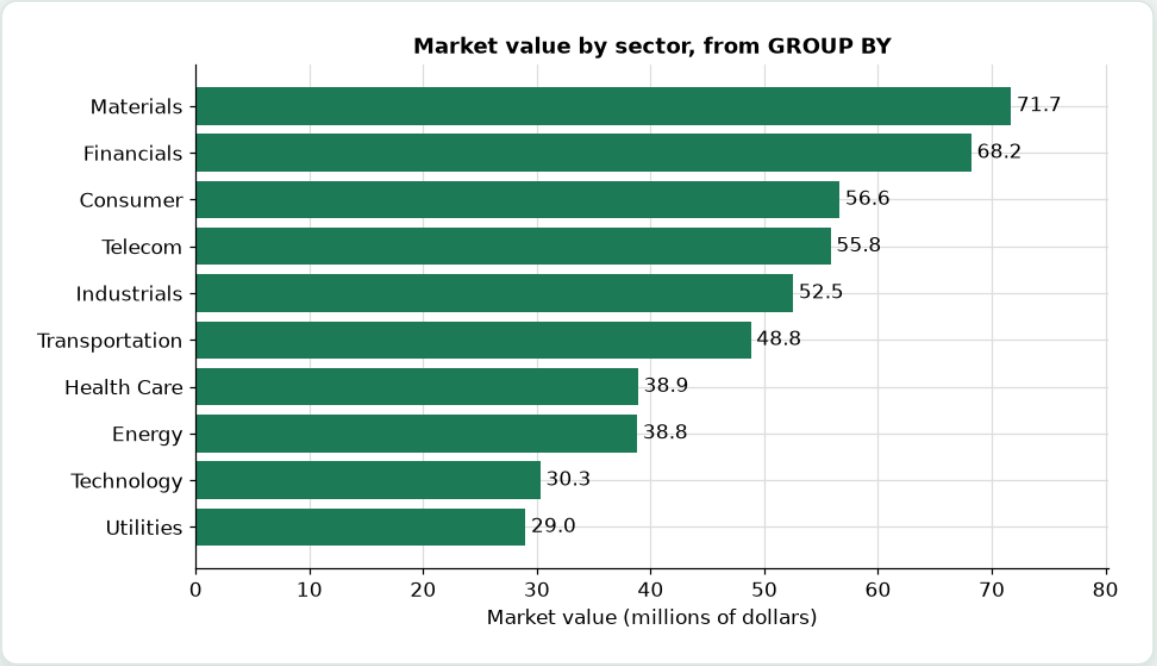
| | |
|---|---|
| `SELECT` | `FROM` |
| `WHERE` | `GROUP BY` |
| `HAVING` | `ORDER BY` |
| `LIMIT` | |

**WHERE filters rows before grouping. HAVING filters groups after. A single row has no COUNT.**

HAVING COUNT(*) >= 20 keeps 7 of 10 sectors. 46 tests pass at the end of the section.

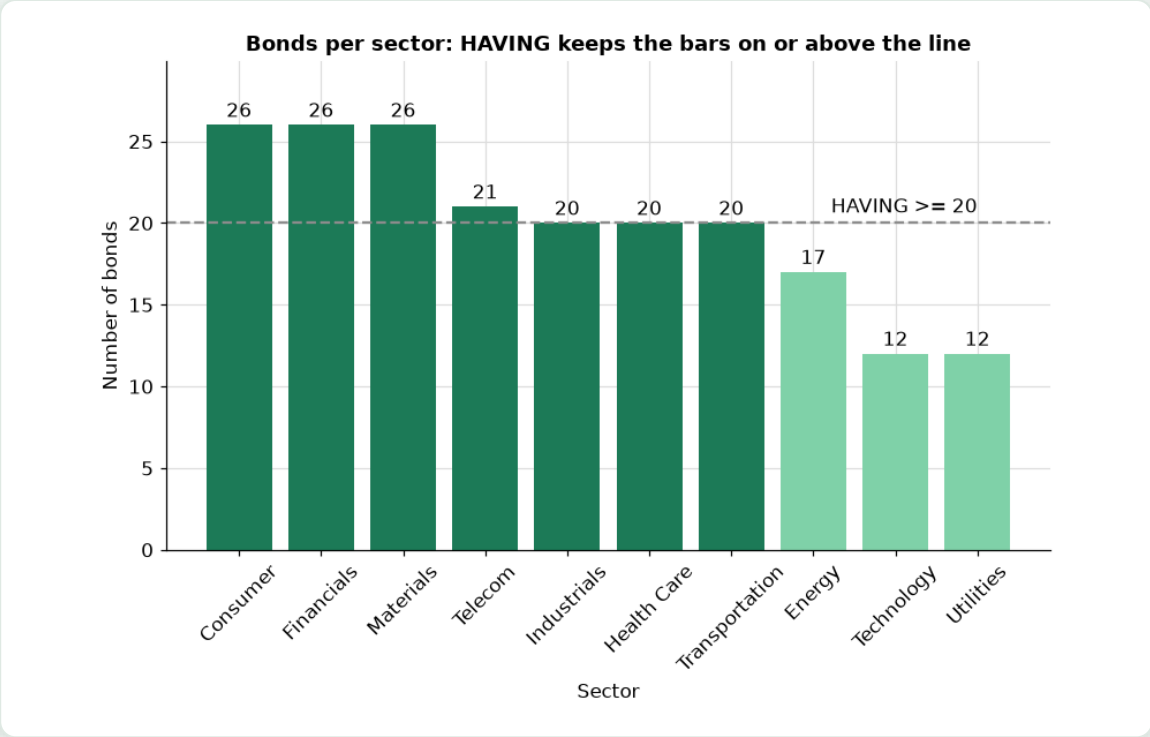
# GROUP BY, drawn

| Sector | Bonds | Market value ($) | Avg OAS (bp) |
|---|---|---|---|
| Materials | 26 | 71,701,618 | 182.42 |
| Financials | 26 | 68,219,672 | 172.31 |
| Consumer | 26 | 56,629,598 | 218.88 |
| Telecom | 21 | 55,848,270 | 182.38 |
| Industrials | 20 | 52,540,745 | 220.55 |
| Transportation | 20 | 48,822,898 | 126.50 |
| Health Care | 20 | 38,908,188 | 141.75 |
| Energy | 17 | 38,830,005 | 171.59 |
| Technology | 12 | 30,311,760 | 298.17 |
| Utilities | 12 | 29,006,462 | 177.92 |

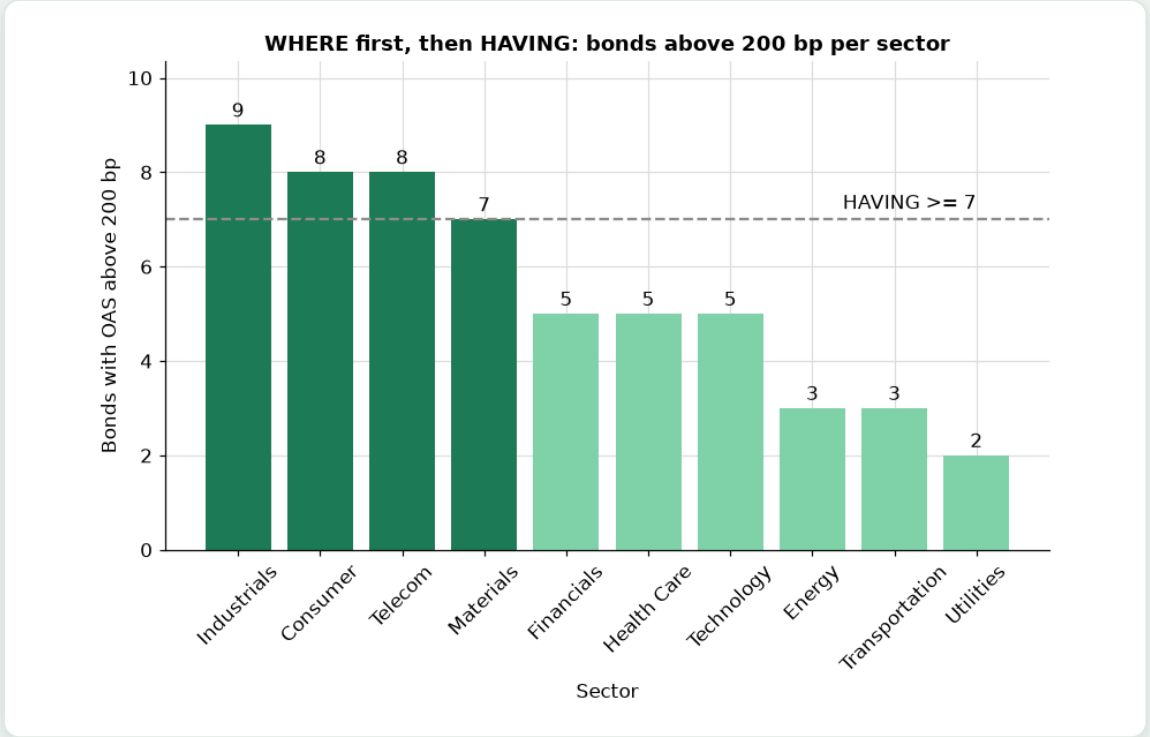



**Styled table: .format is Format Cells, .bar is Data Bars, .background_gradient is Color Scales, .highlight_max is a Top/Bottom rule.**

# HAVING, drawn




HAVING COUNT(*) >= 20 keeps 7 of 10 sectors. Three sit exactly on the line.




WHERE filters rows before grouping; HAVING filters groups after.

# A market-value weighted average in SQL

```
SUM(oas * market_value) / SUM(market_value) AS weighted_oas
```

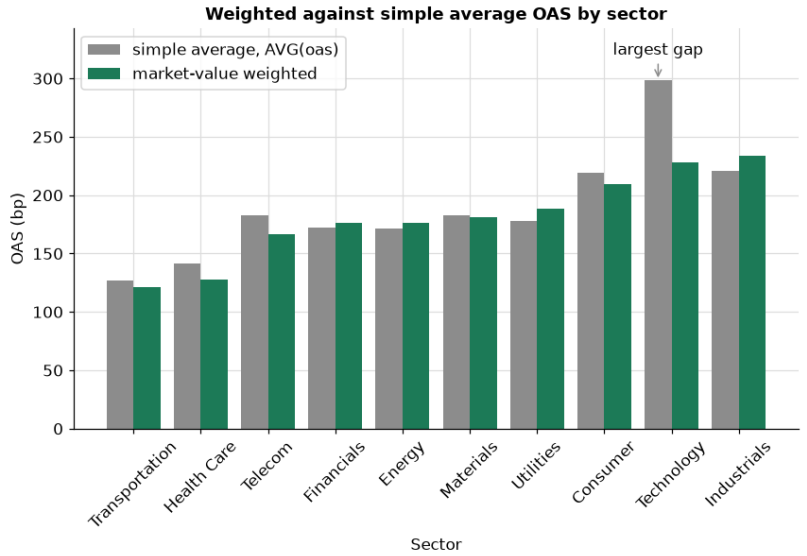
AVG(oas) gives every bond the same say. The weighted average gives every dollar the same say, and bondmath.weighted_average is the answer key.

**180.46**

weighted OAS in bp, 200 bonds (synthetic)

**185.75**

simple AVG(oas) in bp, same bonds




**A query that writes AVG where the question said weighted runs, looks plausible, and is wrong.**

# Excel side by side

| SQL | Excel (tested; long ranges shortened) | Energy |
|---|---|---|
| COUNT(*) … GROUP BY | `=COUNTIFS($C$2:$C$201,J2)` | 17 |
| SUM(market_value) | `=SUMIFS($F$2:$F$201,$C$2:$C$201,J2)` | 38,830,005 |
| AVG(oas) | `=AVERAGEIFS($E$2:$E$201,$C$2:$C$201,J2)` | 171.5882 |
| WHERE … AND oas > 200 | `=COUNTIFS(...,J2,$E$2:$E$201,">200")` | 3 |
| weighted OAS | `=SUMPRODUCT((C=J2)*E*F)/SUMIFS(F,C,J2)` | 176.3494 |

WHERE is FILTER; ORDER BY ... DESC LIMIT 5 is TAKE(SORTBY(...,-1),5); HAVING is FILTER on the summary block.

A pivot table does the whole GROUP BY at once: 10 sectors, Grand Total 200 bonds.

Every formula typed into Excel 16.0 build 20527 by a script on 2026-11-02, equal to the SQL and pandas results.

# Save the day: the database stays out

```
git check-ignore -v course3.db
.gitignore:5:*.db    course3.db
git status --short --ignored
!! course3.db
```

**Committed: four text files**

```
.gitignore
marketdata.py
test_marketdata.py
queries.sql
```

**Binary, rewritten on every load, rebuilt from the CSVs in one call: the CSVs and the code are the record.**

# The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

## Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

## Code back

Written for the course in the style of an AI answer. Read it.

→

3

## Test first

Assert what the docstring promises. Run it: it fails.

→

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: "weighted OAS by grade" written as AVG(oas). It runs and returns plausible spreads; bondmath disagrees.**

*Code that runs is not the same as code that is right.*

# What you can do now

- A SQLite database in one file, kept out of Git with *.db
- load_tables: CSVs to tables, every row count checked
- SELECT, WHERE, ORDER BY, LIMIT, each proved against pandas
- Every value as a ? parameter, never an f-string
- GROUP BY with HAVING; a weighted OAS that bondmath agrees with

**Week 3, Day 2** SQL joins and time series: matching tables, the duplicating join, and dates in SQL